BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

NEST: A Localized Web Application for Enhancing Residential Community Engagement

Supervisor
Dr. Bocicor Maria Iuliana

Author
Bălă Bogdan Andrei

2026

ABSTRACT

Abstract: un rezumat în limba engleză cu prezentarea, pe scurt, a conținutului pe capitole, punând accent pe contribuțiile proprii și originalitate

Contents

1 Project Context and Motivation **1**

1.1 Urban Isolation in Apartment Communities 1

1.2 Local Digital Networks and Mutual Aid 1

1.3 NEST Project Overview 1

2 Technologies and Tools Used **3**

2.1 Frontend Architecture: Angular 3

2.2 State Management: NgRx and Signals 3

2.3 Backend Infrastructure: Node.js and Express 4

2.4 UI/UX Design Principles 5

3 Application Design and Architecture **6**

3.1 System Architecture Overview 6

3.2 Requirements Specification 6

3.2.1 Functional Requirements 7

3.2.2 Non-Functional Requirements 7

3.3 User Roles and Use Cases 8

3.4 Database Schema and API Design 8

4 Implementation Details **10**

4.1 Frontend Structure and Routing 10

4.2 Realizing Functionality F1: User Authentication 10

4.2.1 Implementation of the Login Flow 11

4.2.2 HTTP Interceptors 11

4.3 Authentification 11

4.3.1 Execution and Verification 11

4.4 Implementation of Secondary Modules 14

5 Testing and Validation **15**

5.1 Usability Testing 15

5.2 Cross-Browser Compatibility 15

6 Conclusions and Future Work **16**
6.1 Summary of Contributions 16
6.2 Potential Future Features 16

Bibliography **17**

Chapter 1

Project Context and Motivation

1.1 Urban Isolation in Apartment Communities

In many cities, people live close to each other but interact very little. In apartment blocks, neighbors can share the same building for years without building trust or a real sense of community. This situation is part of a broader decline in social participation and local social capital, described in established sociological research [1].

This lack of connection affects daily life in practical ways: people communicate less, support each other less, and feel less involved in their immediate community. As a result, simple opportunities for socializing and mutual help are often missed.

1.2 Local Digital Networks and Mutual Aid

Research in urban informatics suggests that digital tools can strengthen local communities when they are designed for a specific place and group of people. Local online networks can support, not replace, face-to-face interaction [2].

In this context, a hyper-local platform can act as a small digital meeting point. Residents can share announcements, propose activities, and ask for practical help in a trusted environment [3]. Typical examples include organizing community events, inviting neighbors for casual meetups, coordinating pet care, or borrowing household tools.

1.3 NEST Project Overview

Based on this context, the project proposed in this thesis is NEST, a web application dedicated to residents of the same apartment building. The main goal is to turn online interaction into real-world social activity and everyday collaboration.

The application focuses on three directions: encouraging participation through a clear and friendly interface, supporting community activities through posts and event planning, and enabling mutual aid through targeted local requests. My contribution in this project is the frontend design and implementation of these core flows, together with the integration of backend services needed for authentication and persistent data.

Chapter 2

Technologies and Tools Used

Building a localized web platform capable of handling real-time interaction, structured data flows, and an engaging user experience requires a robust technology stack. This chapter outlines the theoretical background of the primary tools selected for the NEST application, justifying their usage in the context of community-driven software.

2.1 Frontend Architecture: Angular

Modern web development heavily relies on Single-Page Applications (SPAs) to provide a seamless, app-like experience within the browser. SPAs dynamically rewrite the current web page with new data from the web server, instead of the default method of loading entire new pages [4].

For the NEST application, Angular was chosen as the primary frontend framework. Angular provides a structured, component-based architecture that enforces modularity and separation of concerns. This is particularly beneficial for a platform like NEST, where features such as the authentication flow, the community feed, and the mutual aid shed can be developed as isolated, reusable components. Furthermore, Angular's built-in dependency injection system allows for efficient management of services, such as API communication or user state management, ensuring that the application remains scalable as new features are added.

2.2 State Management: NgRx and Signals

Managing the state of a complex web application—such as user session data, loaded feed posts, or pending mutual aid requests—can quickly become difficult to track. Without a predictable state container, data inconsistencies and unexpected side effects frequently occur.

To solve this, the NEST application implements NgRx, a reactive state management framework inspired by Redux [5]. NgRx enforces a unidirectional data flow, where the state is read-only and can only be modified by dispatching actions to reducers.

In addition to traditional NgRx stores, the application leverages Angular Signals, a recent addition to the Angular ecosystem that provides a fine-grained reactivity model. Table 2.1 compares traditional local state management with the Global Store approach used in this project.

Feature	**Component Local State**	**NgRx Global Store + Signals**
Scope	Limited to the specific component.	Accessible application-wide.
Data Flow	Two-way or prop drilling.	Unidirectional and predictable.
Reactivity	Relies on Angular's default change detection (Zone.js).	Fine-grained reactivity; updates only what is necessary.
Use Case	Simple forms, UI toggles.	User sessions, cached feed data, application-wide settings.

Table 2.1: Comparison of State Management Approaches

Justification for NEST: Using NgRx Stores combined with Signals was a deliberate architectural choice for NEST. By decoupling the state from the UI components (the Facade pattern), components remain ”dumb” and strictly focused on presentation. The Store acts as the single source of truth, meaning that if a user claims an item in the Shared Shed, that change is immediately and reactively reflected in their personal dashboard without requiring additional API calls or complex component communication.

2.3 Backend Infrastructure: Node.js and Express

The backend architecture follows a RESTful API design using Node.js and the Express framework [6]. Node.js, built on Chrome's V8 JavaScript engine, allows for non-blocking, event-driven execution. This makes it highly efficient for handling numerous simultaneous requests, a common scenario in community platforms where multiple users might be fetching feed updates or posting messages concurrently.

Express was selected as the web framework due to its minimalist and unopinionated nature, providing robust routing and middleware capabilities. Middleware functions are heavily utilized in the NEST backend for processing incoming

requests, validating data schemas, and enforcing rate limiting to maintain server stability. The Object-Relational Mapping (ORM) is handled by Prisma, which offers type-safe database access, aligning perfectly with the strict typing of the TypeScript-based frontend.

2.4 UI/UX Design Principles

A community application must not only be functional but also deeply intuitive and welcoming to encourage participation [7].

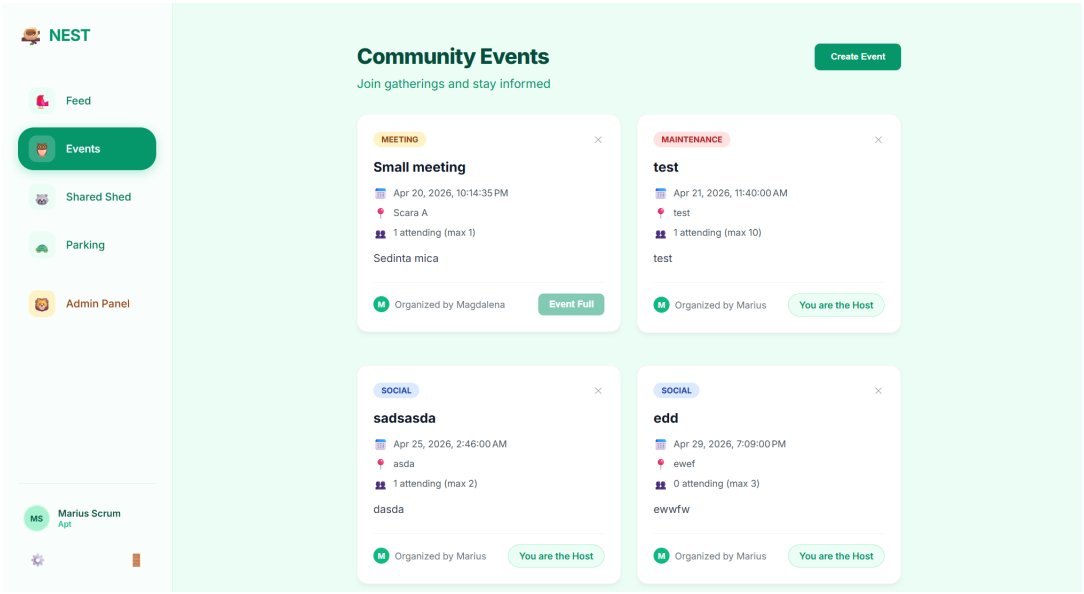

Figure 2.1: Events page of the NEST application.

Key UI/UX decisions include:

- **Vibrant and Accessible Palettes:** Utilizing an emerald-themed palette to convey growth and community trust, while ensuring contrast ratios meet accessibility standards.
- **Progressive Disclosure:** Showing users only the information they need at a given time. For example, detailed item descriptions in the Shed are hidden behind a click or a modal, keeping the main feed uncluttered.
- **Clear Feedback Loops:** Providing immediate visual feedback for user actions (e.g., toast notifications on successful login or item requests) to reduce uncertainty and build trust in the platform.

Chapter 3

Application Design and Architecture

To effectively transform the theoretical goals of community engagement into a practical software solution, a rigorous design and architecture phase is essential. This chapter details the technical specifications, architectural patterns, and functional requirements of the NEST application.

3.1 System Architecture Overview

The NEST platform is built upon a standard Client-Server architectural model. This decoupling of the user interface from the data processing logic ensures that the system is scalable, maintainable, and secure [8].

The frontend client, built with Angular, acts as the presentation layer. It handles all user interactions, local state management, and routing. The backend server, built with Node.js and Express, serves as the central data hub. It exposes a RESTful Application Programming Interface (API) that the client consumes to perform CRUD (Create, Read, Update, Delete) operations.

Data persistence is managed through a relational database, abstracted by the Prisma ORM. Prisma ensures type safety across the database queries, minimizing runtime errors and simplifying the data modeling process.

3.2 Requirements Specification

Defining clear requirements is crucial for guiding the development process and ensuring the final product meets the needs of the target audience.

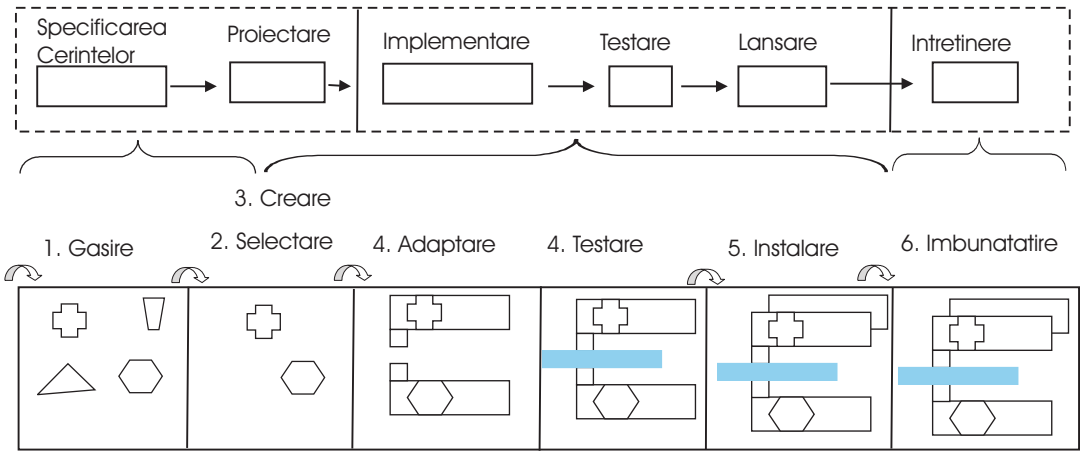

Figure 3.1: System architecture overview of the NEST application.

3.2.1 Functional Requirements

The functional requirements outline the specific behaviors and capabilities the system must provide to the users. These are categorized into core modules in Table 3.1.

Module	Feature	Description
Authentication	User Registration & Login	Users must be able to securely register with an email and password, and log in to receive an authentication token.
Community Feed	Post Creation & Interaction	Residents can create text posts, view a timeline of community updates, and interact (like/comment) with neighbors' posts.
Shared Shed	Resource Borrowing	Users can list household items (tools, appliances) they are willing to lend, and request to borrow items from others.
Events	Community Gatherings	Users can organize local events, set dates, and RSVP to events posted by others in the building.

Table 3.1: Core Functional Requirements of NEST

3.2.2 Non-Functional Requirements

Non-functional requirements dictate how the system should perform under certain conditions:

- **Security:** All API endpoints accessing user data must be protected by JSON Web Token (JWT) authorization. Passwords must be hashed using bcrypt before storage.
- **Performance:** The application must load the initial feed within 2 seconds under normal network conditions.

- **Responsiveness:** The UI must be fully responsive, providing a native-like experience on both desktop and mobile web browsers.

3.3 User Roles and Use Cases

The platform currently supports a flat hierarchy for its primary audience to encourage equal participation, with a distinct role for administration:

- **Resident (Standard User):** Can create posts, list items in the shed, borrow items, RSVP to events, and manage their personal profile.
- **Administrator:** Possesses the same capabilities as a resident but also has moderation rights to remove inappropriate content or ban malicious users from the community space.

Primary Use Case: Borrowing a Tool 1. A resident navigates to the ”Shared Shed” module. 2. They browse or search for a specific tool (e.g., a power drill). 3. Upon finding the tool listed by a neighbor, they click ”Request to Borrow”. 4. The system updates the item’s status to ”Pending” and notifies the owner. 5. The owner approves the request, and the system provides them with a secure channel to arrange the physical handover.

3.4 Database Schema and API Design

The relational database schema is designed around the core entities of the application: `User`, `Post`, `Event`, and `Item`.

The `User` entity has one-to-many relationships with all other entities, representing ownership (e.g., a user can author many posts or own many items). The Prisma schema explicitly defines these relations, which allows the backend to perform complex join queries efficiently.

The API design strictly follows REST principles. For instance, interacting with the Shared Shed utilizes predictable endpoints:

- `GET /api/shed/items`: Retrieves a list of available items.
- `POST /api/shed/items`: Creates a new item listing.
- `PUT /api/shed/items/:id/borrow`: Updates an item’s status to borrowed.

This standardization ensures that the frontend services can easily integrate with the backend without requiring complex custom logic for each action.

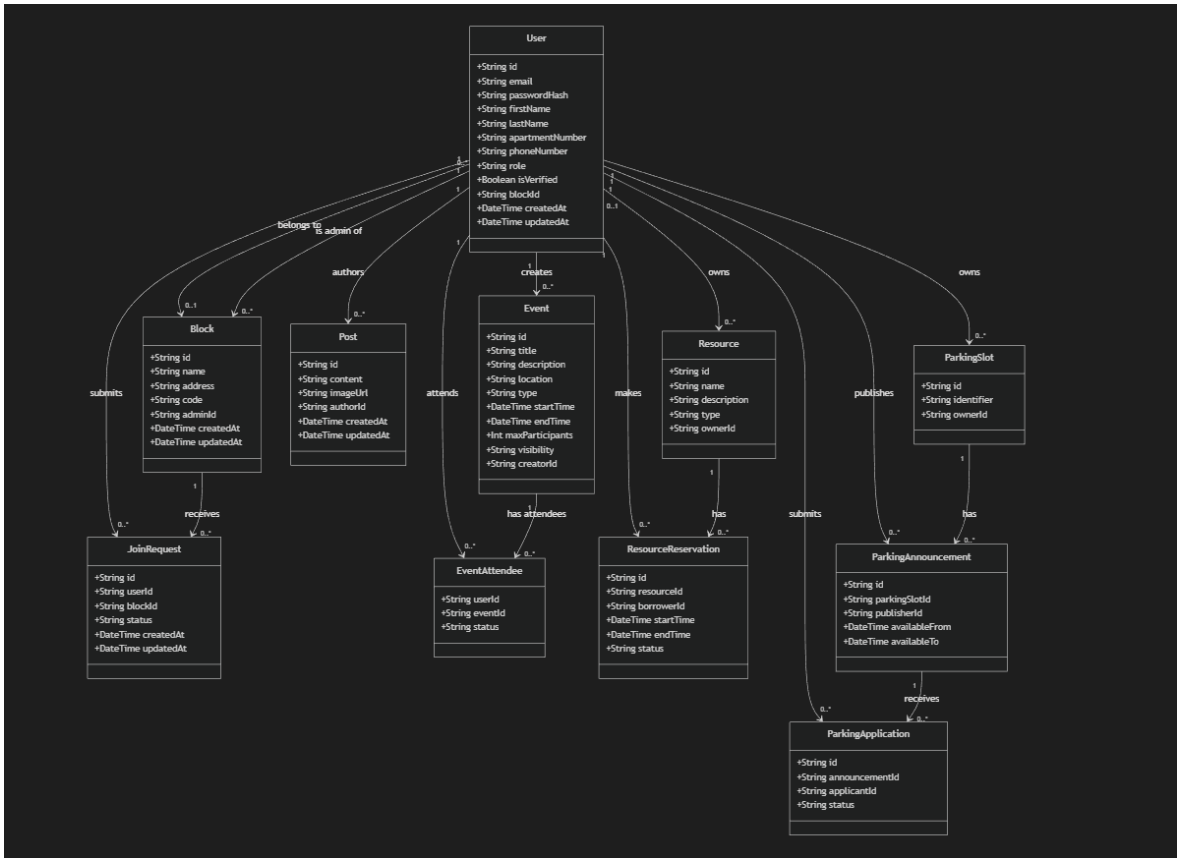

Figure 3.2: UML diagram of the NEST application architecture.

Chapter 4

Implementation Details

Following the architectural design, the implementation phase translates the requirements into functional code. This chapter details the core implementation strategies of the frontend application and provides an in-depth look at the execution and testing of the primary functionality (F1): User Authentication and Profiles.

4.1 Frontend Structure and Routing

The Angular frontend is structured using Domain-Driven Design (DDD) principles. The application is divided into lazy-loaded feature modules: `Auth`, `Feed`, `Shed`, and `Events`. Lazy loading ensures that the browser only downloads the code necessary for the current view, significantly improving the initial load time [4].

Routing is managed by the Angular Router, which binds specific URL paths to their corresponding components. For example, navigating to `/login` renders the `LoginComponent`, while `/dashboard` renders the `DashboardComponent`. To enforce security, Route Guards (`AuthGuard`) are implemented to prevent unauthenticated users from accessing protected routes, redirecting them back to the login page if they lack a valid session token.

4.2 Realizing Functionality F1: User Authentication

User Authentication (F1) is the foundational functionality of the NEST platform, ensuring that community interactions remain secure and attributable to verified residents.

4.2.1 Implementation of the Login Flow

The login process relies on Reactive Forms in Angular to capture user input safely and validate it synchronously (e.g., ensuring the email format is correct and the password meets length requirements) before submission.

When a user submits the form, the `AuthService` dispatches an NgRx action containing the credentials. An NgRx Effect intercepts this action, making an HTTP POST request to the Express backend (`/api/auth/login`). If the backend validates the credentials against the hashed password in the database, it returns a JSON Web Token (JWT). The Effect then dispatches a success action, updating the Global Store with the user's profile and token, and the Router navigates the user to the main feed.

4.2.2 HTTP Interceptors

To maintain the authenticated state across the application, an `AuthInterceptor` is implemented. This service intercepts all outgoing HTTP requests from the frontend and automatically attaches the JWT to the `Authorization` header. This eliminates the need to manually append the token to every single API call, reducing code duplication and preventing security oversights.

4.3 Authentification

Validating the authentication flow is critical. The testing strategy involves both automated unit testing and manual execution verification.

4.3.1 Execution and Verification

To demonstrate authentification as an executable functionality, the following manual test case is performed: 1. **Setup:** The backend server and database are running locally. The Angular development server is launched. 2. **Action:** The user navigates to `localhost:4200/login`, enters valid credentials, and clicks 'Log In'. 3. **Expected Result:** The application briefly displays a loading spinner, the JWT is stored in the browser's `localStorage`, and the view transitions to the Community Feed. A welcome toast notification is displayed. 4. **Negative Test:** The user enters an incorrect password. The system must prevent navigation and display an inline error message ("Invalid credentials") without crashing.

The authentication module includes both the login and registration interfaces shown in Figures 4.1 and 4.2. These views illustrate the entry points used by residents to access the platform and create new accounts.

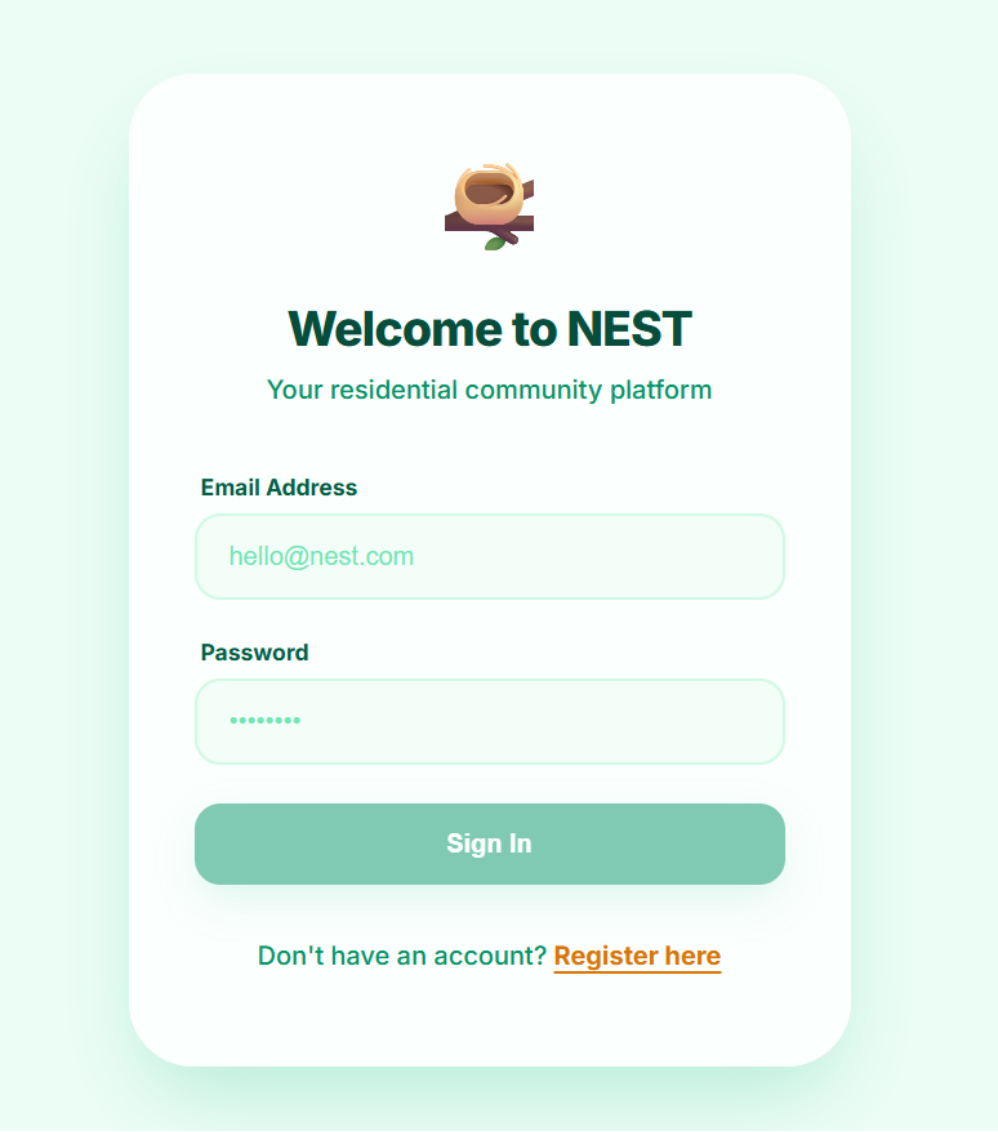

Figure 4.1: Login screen of the NEST application.

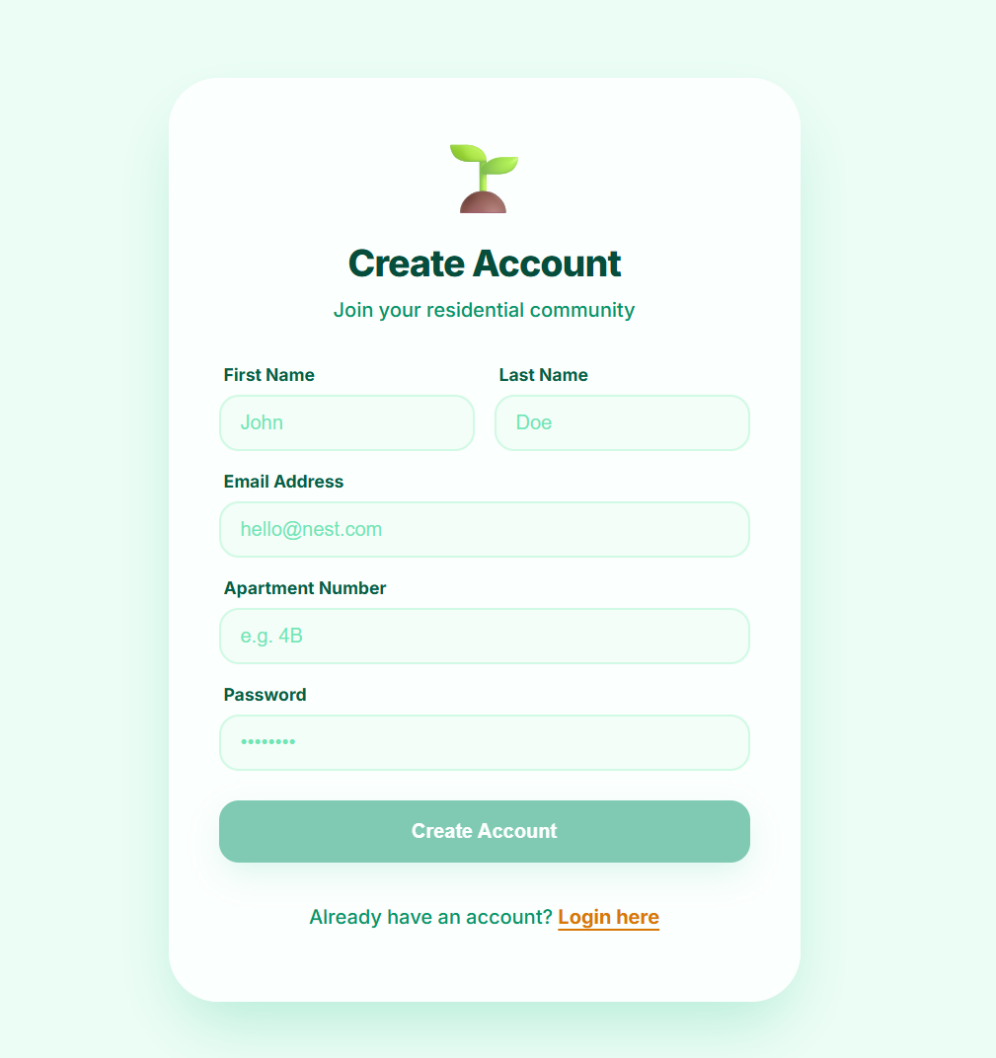

Figure 4.2: Registration screen of the NEST application.

4.4 Implementation of Secondary Modules

While Authentication forms the base, the subsequent modules follow a similar implementation pattern:

- **Community Feed:** Utilizes Angular's `AsyncPipe` to subscribe directly to the NgRx Store, rendering posts dynamically as they arrive without manual subscription management.
- **Shared Shed:** Implements complex reactive forms for item creation, including image upload handling, and communicates with the `ShedApiService` to sync state with the backend database.

Chapter 5

Testing and Validation

5.1 Usability Testing

5.2 Cross-Browser Compatibility

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

6.2 Potential Future Features

Bibliography

[1] Robert D Putnam. *Bowling Alone: The Collapse and Revival of American Community*. Simon and Schuster, 2000.

[2] Keith Hampton and Barry Wellman. Neighboring in netville: How the internet supports community and social capital in a wired suburb. *City & Community*, 2(4):277–311, 2003.

[3] Christina A Masden, Catherine Grevet, Rebecca E Grinter, Eric Gilbert, and W Keith Edwards. Tensions in scaling-up community social media: A multi-neighborhood study of nextdoor. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 3239–3248, 2014.

[4] Google LLC. Angular Documentation. `https://angular.io/docs`, 2024. Accessed: 2026-05-04.

[5] NgRx Team. NgRx Documentation. `https://ngrx.io/docs`, 2024. Accessed: 2026-05-04.

[6] OpenJS Foundation. Express.js Documentation. `https://expressjs.com/`, 2024. Accessed: 2026-05-04.

[7] Donald A Norman. Cognitive engineering. *User centered system design: New perspectives on human-computer interaction*, 31(61):31–61, 1986.

[8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.